

AJX热更新

1. 背景

1.1 AJX热更新介绍



如上图所示，AJX引擎中biz-engine动态引擎是一个重要的模块，该模块主要负责发布单的拉取，SO、APK、AJX、资源等bundle的下载，AJX Bundle的热加载，缓存文件的管理等；服务于业务五毒快迭发布，基于SO、AJX等热更新问题修复，以及整个AJX页面的加载等一系列工作，因此在整个APP中起着至关重要的作用。

AJX Bundle热更新是业务基于五毒快迭发布或者基于AJX发布单的形式进行问题修复的一种能力。由动态引擎负责拉取发布单更新(冷启动拉取发布单更新、场景化进页面拉取发布单更新、scheme跳转拉取发布单更新)、文件下载、校验、加载等一系列工作，最后AJX Bundle在端上进行生效。

1.2 业务问题

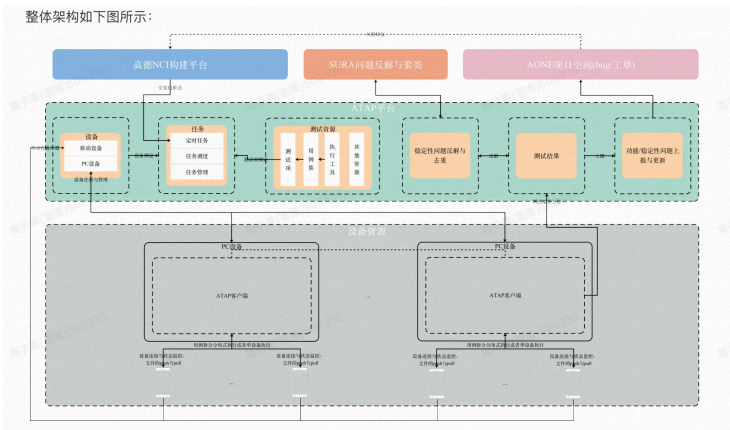
- 在版本节奏不变的前提下，随着动态引擎通道与能力的不断扩展，以及新需求的不断膨胀，进行动态引擎基础功能回归的成本越来越高，且对QA人力的挑战越来越大；
- 测网环境等不稳定因素，使得测试无法进行，进而阻塞测试的执行；
- 基于梵天平台发布单的不断创建、发布单的不断切换，以及相同bundle间不同版本之间的相互干扰，会延长回归时间，甚至导致封板前不能按时完成测试，进而加大线上隐患；

基于上述问题，现亟需建设动态引擎的热更新自动化测试能力，从而在保障质量的前提下即屏蔽外界环境等因素的干扰，又能够提升测试效率。

2. 方案

2.1 整体架构

整体架构如下图所示：



- **titan/nci构建平台**：测试过程中，titan/nci构建平台主要进行bug修复后的代码集成、Android端与iOS端APP安装包的构建、aone空间的获取；
- **aone bug/工单**：负责bug或者工单的上报；
- **atap测试平台**：进行测试设备的调度，测试任务的触发与调度执行，执行工具、用例集、测试项、测试资源的管理，测试报告的模板配置与展示、bug/工单的模板配置与展示、bug/工单的上报等；
- **atap客户端**：测试设备注册、客户端文件缓存等；
- **工具执行包**：测试工具的产物为jar包，主要负责驱动Totoro测试脚本，以及获取测试结果，并进行测试结果的处理与上报；
- **测试脚本包**：测试脚本有两种，一种是基于Totoro的jar包，主要通过UI自动化的方式进行APP安装后权限的验证并进入到AMAP首页，然后基于totoro驱动测试设备执行AJX Bundle拉取热更新能力，并进行结果断言；另外一种是在端内的ajx测试bundle，产物形式为app内ajx文件，主要拥挤通过ajx的切面接口进行页面跳转，从而触发bundle更新能力；
- **测试设备**：主要分为Android设备与iOS设备，用于执行端内的测试，由Totoro脚本驱动执行；

2.2 详细方案

2.2.1 端内测试通道

端内触发bundle热更新后，实际上是请求服务端的接口来获取bundle更新能力，服务返回的接口数据实际上是从梵天平台创建发布单进行下发的，形式如下：



复制代码

```

{
  "code": 1,
  "message": "success",
  "status": 1, //数字, 1、有更新, 此时data不为空; 2、当前没有更新(客户端上
传dversion与服务端dversion相同), 此时data为空;
  "dversion": "10001", //字符串, 当前服务端数据版本, 下次请求原样带给服务端
  "timestamp": 1231231, //数字, 当前服务端时间, 用于时间校准。单位: 秒;
  "data": {
    "groups": [
      {
        "bundles": [
          {
            "name": "amap_bundle_feedback", //资源名, 字符串
            "resId": "A1234", //资源ID, 字符串, 平台为所有资源
            进行ID编排, 资源ID全局唯一, 一旦确定后, 永久不可更改, 资源ID第一位为字母, 代表
            不同的资源类型; A: ajx; S: so; K: apk; Z: zip, O: other 其他
            "type": 1, //资源类型, 数字, 1、AJX; 2、SO; 3、
            APK; 4、zip; 5、7zip; 6、数据文件; 999、其他
            "size": 1200, //资源包大小, 数字, 单位 B
            "md5": "cfa0a51523146ba4945a0de1ec26d366", //若当
            前资源为diff, 则表示资源diff的md5值, 否则表示资源的整包md5值
            "sign":
            "Xw9uOdYo1NJ6+p5RiTiuPUts+QgWueVS2xHFeb3qpsr+/Ejyj37jA4YYAX1zEKMhLF6RrTF
            SHQB57takcAG32RVYWV3srcZSGi8JMAeaG9FCiUCLKKJCjbtLWnJO+NFxW+e/ZIqWJWkESE
            KoBZ2rt7pXMXo3bQkjyxRc3AeGac=", //若当前资源为diff, 则表示资源diff的签名,
            否则表示资源整包的签名
            "version": "1.1.0.1123", //当前下发版本号
            "reqVersion": "1.0.0.1123", //客户端请求时版本号
            "url":
            "http://cache.amap.com/test/ajx/pkg/0011/100700/b087d891d4144a79a8749f3c
            e205b500/amap_bundle_feedback_1.0.0.1123.ajx", //若当前资源为diff, 则表示
            资源diff的url, 否则表示资源整包的url
            // 若当前资源为diff, 则会给出整包的信息, 否则无
            targetInfo字段
            "targetInfo": {
              //给出整包md5和下载URL
              "md5": "cfa0a51523146ba4945a0de1ec26d366",
              //整包的md5值
              "url":
              "http://cache.amap.com/test/ajx/pkg/0011/100700/b087d891d4144a79a8749f3c
              e205b500/amap_bundle_feedback_1.0.0.1123.ajx", //整包的下载url
              "sign":
              "Xw9uOdYo1NJ6+p5RiTiuPUts+QgWueVS2xHFeb3qpsr+/Ejyj37jA4YYAX1zEKMhLF6RrTF
              SHQB57takcAG32RVYWV3srcZSGi8JMAeaG9FCiUCLKKJCjbtLWnJO+NFxW+e/ZIqWJWkESE
              KoBZ2rt7pXMXo3bQkjyxRc3AeGac=" //整包的签名
            }
          }
        ]
      }
    ]
  }
}

```

```

    },
    "resType": 0, //包类型, 数字, 0、整包; 1、Diff 包
    "env": "1|2|3", //下载时机, 可以多选, 用|分隔; 1、
冷启动; 2、热启动; 3、使用时按需下载;
    "priority": 1, //下载优先级, 数字, 1、高优(立即)下
载; 2、中优下载; 3、低优下载;
    "network": 1, //下载网络, 数字, 1、全网络; 2、
WIFI; 3、数据网络;
    "library": 0, //是否为基础库, 0: 非基础库; 1: 基础
库;
    "restartService": 1, //是否重启js Service, 数字;
0: 不重启; 1: 重启;
    "minAppVersion": "1000705", //最小生效app版本号
    "minBundleVersion": "1.1.0.1123", //最小生效
bundle版本号

    "dependencies": [
        {
            "name": "amap_bundle_framework",
            "resId": "A1234",
            "version": "1.1.0.1245"
        }
    ]
}
// ...
}, {
},
],
"actions": []
}
}

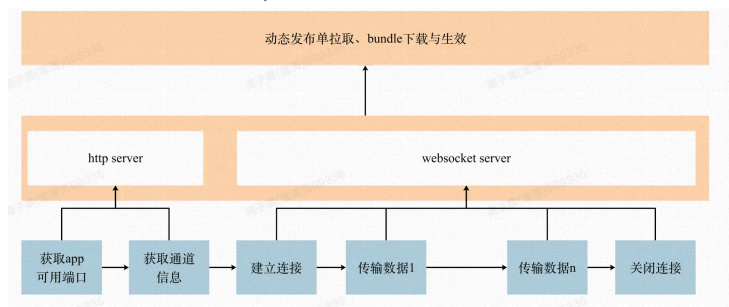
```

但是, 目前存在如下几个问题:

- 发布单依赖手动创建;
- 需要频繁切换发布单;
- 创建后各个测试用例之间存在干扰, 导致测试结果不够准确;
- 测网环境不稳定阻塞用例执行;

基于以上问题, 构建了一个端内基于HTTP与WebSocket的测试通道, 通过该通道将测试的发布单数据传递给biz-engine动态引擎。debug环境下ajx-engine中内置了http server与

websocket server，因此可以基于这个能力来建立测试通道，整体流程如下图所示：



1. 获取通道内可用的socket端口

`http://{device_ip}:{port}/info`

端口的范围为8888~8892，因此需要轮询端口，若端口可用则会返回如下的JSON信息，其中包括了设备adiu与diu信息、app版本号、AJX引擎信息等；

复制代码

```
{
  "Protocol-Version": "1.1",
  "adiu": "keaiiiffdf886ij78666676f1c0d6d",
  "appIsDebuggable": false,
  "appVersion": "11.18.0.1731",
  "deviceDebugable": true,
  "deviceName": "iPhone%208Plus",
  "deviceOS": "iOS_15.5",
  "diu": "CDB1494F-8807-4747-925E-D36493E84B54",
  "engineVersion": "11.18.0.65"
}
```

2. 获取WebSocket通道信息

`http://{device_ip}:{port}/json`

其中的端口为上一步获取的可用端口，该服务会返回如下的JSON信息，其中title为biz_engine_auto_test的JSON对象为当前热更新的WebSocket通道信息；

复制代码

```
[
  {
    "devtoolsFrontendUrl": "http://chrome-devtools-frontend.appspot.com/serve_rev/@186555/devtools.html?ws=30.28.176.239:8888/devtools/page/671B44E3-6D63-4162-A561-D10E59BE9915",
    "id": "671B44E3-6D63-4162-A561-D10E59BE9915",
    "title": "biz_engine_auto_test",
    "type": "extend_service",
    "websocketDebuggerUrl": "ws://30.28.176.239:8888/devtools/page/671B44E3-6D63-4162-A561-D10E59BE9915"
  }
]
```

```
D10E59BE9915"
  },
  {
    "devtoolsFrontendUrl": "http://chrome-devtools-frontend.appspot.com/serve_rev/@186555/devtools.html?ws=30.28.176.239:8888/devtools/page/4E8CC3A6-FE49-4702-A10A-E94B79B45167",
    "id": "4E8CC3A6-FE49-4702-A10A-E94B79B45167",
    "title": "fast_refresh_service",
    "type": "extend_service",
    "websocketDebuggerUrl":
      "ws://30.28.176.239:8888/devtools/page/4E8CC3A6-FE49-4702-A10A-E94B79B45167"
  }
}
```

3. 建立WebSocket连接

连接地址为上一步中的websocketDebuggerUrl;

4. 传输数据

通过WebSocket通道传输数据给biz-engine引擎，数据格式如下：

复制代码

```
{
  "id": XXX,
  "method": "autotest.XXX",
  "params": {
    "data": {
      // ...
    }
  }
}
```

- id: Integer类型，表示当前的请求ID，自定义设置；
 - method: String类型，表示请求方法；
- WebSocket server返回的数据格式如下：

复制代码

```
{
  "id": XXX,
  "result": {
    "errstring": "success",
    "data": {
      // ...
    }
  }
}
```

}

- id: 与请求ID一致, 用于校验;
 - result.errstring: 成功则返回"success";
 - result.data: 与请求对应的响应数据;
- method、请求参数与响应参数如下:

method	方法说明	params.data参数	响应
autotest.init	模拟init服务接口返回的数据, 直接将数据写入init.txt文件, app下次冷启动或者1小时外前后台切换时, 从init接口请求回数据后, 将此文件数据与init接口返回数据进行merge(同一bundle进行覆盖), 并删除init.txt文件	发布单	{"id":1,"result":{"data":{},"errstring":"success"}}
autotest.scene	模拟scene服务接口返回的数据, 直接将数据merge到服务表(同一bundle进行覆盖)	发布端	{"id":2,"result":{"data":{},"errstring":"success"}}
autotest.scheme	autotest.scheme 模拟scheme服务接口返回的数据, 直接将数据merge到服务表(同一bundle进行覆盖)	发布单	{"id":3,"result":{"data":{},"errstring":"success"}}

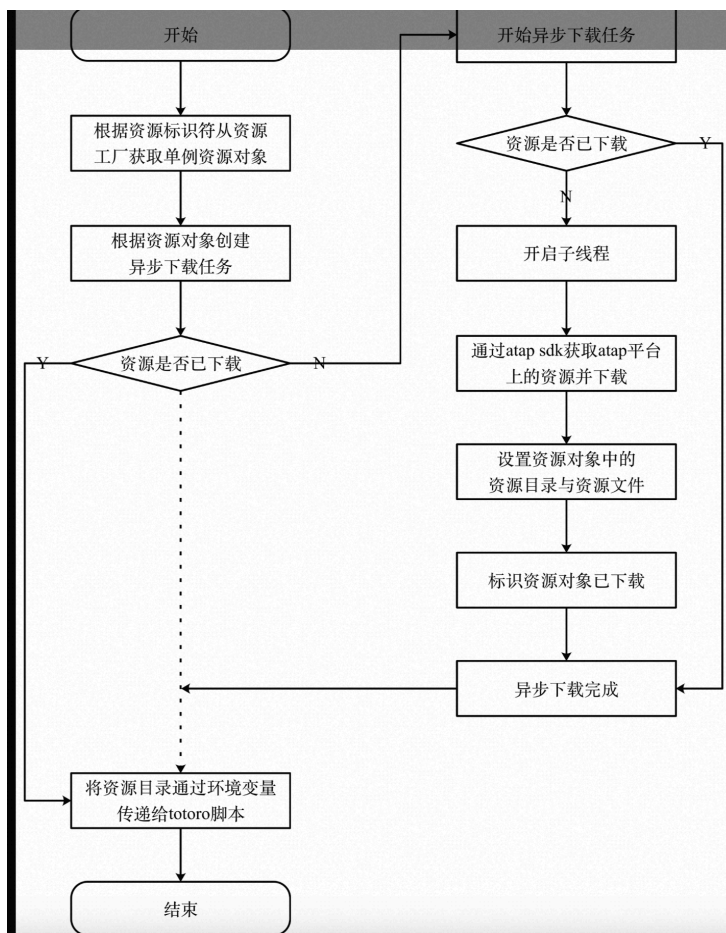
autotest.rollback	模拟服务回滚数据，直接将数据写入 rollback.txt 文件，app 下次冷启动或者前后台切换时，从云控服务接口请求回数据后，将此文件数据覆盖云控接口返回数据，并删除 rollback.txt 文件	回滚配置，如 {"ajx_rollback": {"status":1,"version":"F0C8E8C84C7859CFD6FD698BFCA9A031","value": {"notice_content":"数据已更新，即将重启高德地图","restart_type":0,"rollback_all":1,"id":23412,"value": ["amap_bundle_testSchemeKeyWeb","amap_bundle_module_auto_test"]}"]}}	{"id":4,"result":{"data": {}, "errstring":"success" }}
autotest.get_timestamp	获取当前的内存时间戳	无	{"id":5,"result":{"data": {"timestamp":1660543142}, "errstring":"success"}}
autotest.set_timestamp	设置内存时间，此时参数为时间戳(秒级别)	时间戳，如 {"timestamp": 1660542560}	{"id":6,"result":{"data": {}, "errstring":"success" }}
autotest.enter_foreground	进入前台事件	无	{"id":7,"result":{"data": {}, "errstring":"success" }}

数据传输完毕后，关闭测试通道；

2.2.2 测试用例执行过程

测试执行工具的整体执行流程如下图所示：

- **atap** 执行包会解析 XML 用例集，然后按照 **testNG** 的形式进行组织测试用例
 - 对于 AJAX Bundle 的热更新自动化测试，测试工具包在执行测试任务前需要下载前置的测试资源，主要包括 ajax 解码工具等资源，均存储在 ATAP 平台上，具体流程如下：



- 驱动totoro的执行：主要通过mvn命令来执行JUnit TestCase



复制代码

```

IScriptExecutor scriptExecutor =
ScriptExecutorFactory.getExecutor(testCase.getScriptTypeEnum());
scriptExecutor.execute(testConfig, testCase,
TestContext.getScriptResult());
  
```

• totoro测试脚本主要进行测试用例的执行

测试脚本的整体执行流程如下所示：

- totoro执行前需要进行上下文的初始化；
- 执行AJX测试用例前，需要获取通过atap执行工具传递的环境变量参数；
- 执行测试用例，包括将mock发布单数据传递给动态引擎，触发热更新并获取对应的端内动态引擎数据、解码、服务请求的获取测试资源中的mock数据格式如下，其中的mock_order_info中的键被称为TAG，order_list对应当前测试用例所需要的bundle发布单及其发布条件：

在获取发布单数据时，在不破坏原始发布单的前提下需要根据当前测试用例中对应mock数据的发布单进行原始发布单的组合，以便其它用例能够再次复用原始发布单，降低发布单数据的维护成本。

用例执行完毕，进行结果的断言：主要根据基线表中bundle的版本、服务表中bundle的版本、生效表中bundle的版本、彩蛋中被加载的bundle版本进行判断：

```
try {
    String bundleName = "amap_bundle_testSchemeKeyWeb";
    // 1. 从mock中获取Bundle信息
    BundleOrderParser bundleOrderParser = BundleOrderParser.getParser();
    BundleOrderEntity bundleOrderEntity =
bundleOrderParser.parseBundleOrder(mock.getMockResponse());
    boolean bundleOrderIsEmpty = bundleOrderEntity.isEmptyBundleOrder();
    Assert.assertFalse("bundle order is not an empty order.",
!bundleOrderIsEmpty);
    // 2. 从彩蛋中获取信息
    ColorEggMetaInfo colorEggMetaInfo = (ColorEggMetaInfo)
colorEggInfo.getBundleInfoByBundleName(bundleName);
    Assert.assertFalse("bundleLoadInfo is null", colorEggMetaInfo ==
null);
    // 3. 从生效表中获取信息(获取动态引擎的生效表数据并解码)
    BundleTableInfo activeItem = (BundleTableInfo)
activeTable.getBundleInfoByBundleName(bundleName);
    Assert.assertFalse("activeItem is null", activeItem == null);
    // 4. 从服务表中获取信息(获取动态引擎的服务表数据并解码)
    BundleTableInfo serverItem = (BundleTableInfo)
serverTable.getBundleInfoByBundleName(bundleName);
    Assert.assertFalse("serverItem is null", serverItem == null);
    // 5. 从基线表中获取信息(获取动态引擎的基线表数据并解码)
    BundleTableInfo baseLineItem = (BundleTableInfo)
baseLineTable.getBundleInfoByBundleName(bundleName);
    Assert.assertFalse("app package error, keyWeb baseline is not
existed.", baseLineItem == null);
    IVersionParser<VersionEntity> versionParser =
VersionParser.getParser();
    VersionEntity loadedBundleVersion =
versionParser.parse(colorEggMetaInfo.getValue());
    // 6. 比较彩蛋与生效表中bundle版本是否相同
    VersionEntity activeBundleVersion =
versionParser.parse(activeItem.getVersion());
    int compareResult =
loadedBundleVersion.compareVersion(activeBundleVersion);
    Assert.assertFalse(String.format("loadedBundleVersion(%s) is not
equal to activeBundleVersion(%s)", loadedBundleVersion,
activeBundleVersion), compareResult != 0);
    // 7. 比较彩蛋中bundle版本是否等于基线表中bundle版本
    VersionEntity baseLineBundleVersion =
versionParser.parse(baseLineItem.getVersion());
    compareResult =
loadedBundleVersion.compareVersion(baseLineBundleVersion);
```

```

        Assert.assertFalse(String.format("loadedBundleVersion(%s) is not
equal to baseLineBundleVersion(%s)", loadedBundleVersion,
baseLineBundleVersion), compareResult != 0);
        // 8. 比较彩蛋中bundle版本与服务表中版本是否相同
        VersionEntity serverBundleVersion =
versionParser.parse(serverItem.getVersion());
        compareResult =
loadedBundleVersion.compareVersion(serverBundleVersion);
        Assert.assertFalse(String.format("loadedBundleVersion(%s) is not
equal to serverBundleVersion(%s)", loadedBundleVersion,
serverBundleVersion), compareResult != 0);
        // 9. 断言网络接口 => 仅请求scheme接口，不携带当前资源
        boolean amockRecordAssertResult =
assertAmockRecordRequestWhenNotLoaded(bundleName,
ConstantConfig.SchemeConfig.KEYWEB_SCHEME_MAPPING_KEY,
ajaxAmockRecordList, activeItem.getResId());
        Assert.assertFalse("amockRecordAssertResult fail",
!amockRecordAssertResult);
    } catch (Exception e) {
        e.printStackTrace();
        Assert.assertFalse(e.getMessage(), true);
    }
}

```

最后将测试结果及相关的数据进行本地存储，其中testResult结构如下：

复制代码

```

public class AjaxHotLoadTestResultResource {
    private String testId;
    private String testCase;
    private List<TestResultResourceMock> testResultResourceMockList;
    private List<String> screenFileList;
    private Set<String> alcSet;
    private WsAppDebugInfo wsAppDebugInfo;
    private List<String> bizEngineTablePathList;
    private String atapWorkPath;
    private List<AjaxHotloadAMockRecordEntity>
amockRecordFilterResultList;
}

```

最终的测试结果文件如下所示：

复制代码

```

{
    "bizEngineBaseLineTableAtap": "/Users/snowy/atap-
client/runtime/1074668/AJX动态热更新自动化测

```

```

    试/ajx_bundle_hotload_totoro/ajx_hotload_totoro_results_1675940085978/tc
    _002_004_001/biz_engine_table/base.json",
    "wsTestChannelInfo": "/Users/snowy/atap-client/runtime/1074668/AJX动态
    热更新自动化测
    试/ajx_bundle_hotload_totoro/ajx_hotload_totoro_results_1675940085978/tc
    _002_004_001/ws_test_channel.json",
    "bizEngineActivesTableAtap": "/Users/snowy/atap-
    client/runtime/1074668/AJX动态热更新自动化测
    试/ajx_bundle_hotload_totoro/ajx_hotload_totoro_results_1675940085978/tc
    _002_004_001/biz_engine_table/actives.json",
    "bizEngineServerTableAtap": "/Users/snowy/atap-
    client/runtime/1074668/AJX动态热更新自动化测
    试/ajx_bundle_hotload_totoro/ajx_hotload_totoro_results_1675940085978/tc
    _002_004_001/biz_engine_table/server.json",
    "mockDataPathList": [
        "/Users/snowy/atap-client/runtime/1074668/AJX动态热更新自动化测
        试/ajx_bundle_hotload_totoro/ajx_hotload_totoro_results_1675940085978/tc
        _002_004_001/mock.t.json"
    ],
    "resultAlcZipFile": "/Users/snowy/atap-client/runtime/1074668/AJX动态热
    更新自动化测
    试/ajx_bundle_hotload_totoro/ajx_hotload_totoro_results_1675940085978/tc
    _002_004_001/hot_load_ajx_alc.zip",
    "colorEggInfoAtap": "/Users/snowy/atap-client/runtime/1074668/AJX动态热
    更新自动化测
    试/ajx_bundle_hotload_totoro/ajx_hotload_totoro_results_1675940085978/tc
    _002_004_001/biz_engine_table/color_egg_info.txt",
    "testCase": "const options = {\n\tmode: \"by_name\", \n\ttparams:
    {\n\t\tkey: \"...\", \n\t\taction: \"background\" \n\t} \n}; \nconst req_id
    = ajx.cloudbundle.download(options, (progress) => {\n\t// ... \n},
    (success, message) => {\n\t// ... \n}); \najx.cloudbundle.cancel(req_id); \n
    前置条件: \n(1) 端侧网络正常; \n(2) 发布单下发biz更新, 配置\"异步刷新\";
    \n(3) 本地不存在目标biz; \n(4) app不存在biz内置信息; \n(5) 服务表无更新; \n操
    作步骤: \n(1) 调用ajx module接口下载biz; \n预期1: 发送scene请求; \n预期2: 下
    载成功并生效; ",
    "resultScreenShotFiles": [
        "/Users/snowy/atap-client/runtime/1074668/AJX动态热更新自动化测
        试/ajx_bundle_hotload_totoro/ajx_hotload_totoro_results_1675940085978/tc
        _002_004_001/screen_shot/tc_002_004_001_service_test.jpg"
    ]
}

```

2.3 接入详情

1. 执行工具

- 对于atap执行工具，通过mvn来构建jar包并上传到执行工具空间



 复制代码

```
mvn clean package -U -DskipTests
```

- 执行工具空间



其中，parameter.xml为执行参数，具体如下所示：

 复制代码

```
<?xml version="1.0" encoding="UTF-8"?>
<atap-configuration>
  <parameters>
    <parameter name="reportStability" description="上报奔溃信息"
optional="true" text="上报崩溃信息" value-type="CheckBox" value="true"/>
    <parameter name="collectLogalc" description="收集alc日志"
optional="true" text="收集alc日志" value-type="CheckBox" value="true"/>
    <parameter name="enableScreenshotTimeLine" description="是否有操
作事件截图" optional="true" text="操作事件截图" value-type="CheckBox"
value="true"/>
    <parameter name="enableLogcat" description="自动化任务过程中收集日
志" optional="true" text="收集logcat日志" value-type="CheckBox"
value="true"/>
    <parameter name="keepTotoroProcessBeforeAfterCase"
description="用例执行前后不killtotoro进程" optional="true" text="用例执行前
后不killtotoro进程" value-type="CheckBox" value="true"/>
    <parameter name="video_switch" value="true" text="是否开始视频录
制" description="是否开始视频录制" optional="true" value-
type="ToggleButton"/>
    <parameter name="clearContextSwitch" value="false" text="测试完成
后清理文件" description="测试完成后清理文件" optional="true" value-
type="ToggleButton"></parameter>
  </parameters>
</atap-configuration>
```

2. 脚本包与用例集

- 脚本包也通过mvn命令来构建，产物行为仍然为jar包

[复制代码](#)

```
mvn clean package -U -DskipTests
```

用例集空间



其中，用例集配置文件为标准的totoro用例集文件，如下所示：

[复制代码](#)

```
<?xml version="1.0" encoding="UTF-8"?>
<atap-configuration>
  <test-cases>
    <test-cases name="com.totoro.auto.testcases.ajx.hotload.test">
      <test-cases name="AjxHotLoadInitTest">
        <test-case summary="001_001" type="totoro" bizline="高德" author="宝森"
          atapCaseMethodName="com.totoro.auto.testcases.ajx.hotload.test.AjxHotLoadInitTest#TC_001_001" name="TC_001_001#AjxHotLoadInitTest#app冷启动动态引擎测试" ajx_test_type="5" timeoutInMillis="1800000"
          description="001_001[app冷启动动态引擎测试]" priority="P0"
          platform="android,ios" groupName="高德"
          scriptJar="AmapContainerArchTotoro-1.0.0-SNAPSHOT-tests.jar"/>
        <!-- ... -->
      </test-cases>
      <test-cases name="AjxHotLoadSceneTest">
        <test-case summary="002_001_001" type="totoro"
          bizline="高德" author="宝森"
          atapCaseMethodName="com.totoro.auto.testcases.ajx.hotload.test.AjxHotLoadSceneTest#TC_002_001_001" name="TC_002_001_001#AjxHotLoadSceneTest#app
          端内进页面动态引擎测试" ajx_test_type="5" timeoutInMillis="1800000"
          description="002_001_001[app端内进页面动态引擎测试]" priority="P0"
          platform="android,ios" groupName="高德"
          scriptJar="AmapContainerArchTotoro-1.0.0-SNAPSHOT-tests.jar"/>
        <!-- ... -->
      </test-cases>
      <test-cases name="AjxHotLoadSchemeTest">
```

```

        <test-case summary="002_002_001" type="totoro"
bizline="高德" author="宝森"
atapCaseMethodName="com.totoro.auto.testcases.ajax.hotload.test.AjxHotLoa
dSchemeTest#TC_002_002_001"
name="TC_002_002_001#AjxHotLoadSchemeTest#app端内scheme跳转动态引擎测试"
ajax_test_type="5" timeoutInMillis="1800000" description="002_002_001[app
端内scheme跳转动态引擎测试]" priority="P0" platform="android,ios"
groupName="高德" scriptJar="AmapContainerArchTotoro-1.0.0-SNAPSHOT-
tests.jar"/>

        <!-- ... -->
    </test-cases>
    <test-cases name="AjxHotModuleDownloadTest">
        <test-case summary="002_004_001" type="totoro"
bizline="高德" author="雪平"
atapCaseMethodName="com.totoro.auto.testcases.ajax.hotload.test.AjxHotMod
uleDownloadTest#TC_002_004_001"
name="TC_002_004_001#AjxHotModuleDownloadTest#ajax module by_name接口直接
下载动态引擎测试" ajax_test_type="5" timeoutInMillis="1800000"
description="002_004_001[ajax module by_name接口直接下载动态引擎测试]"
priority="P0" platform="android,ios" groupName="高德"
scriptJar="AmapContainerArchTotoro-1.0.0-SNAPSHOT-tests.jar"/>

        <!-- ... -->
    </test-cases>
    <test-cases name="AjxHotLoadOuterDevice">
        <test-case summary="002_005_002" type="totoro"
bizline="高德" author="宝森"
atapCaseMethodName="com.totoro.auto.testcases.ajax.hotload.test.AjxHotLoa
dOuterDevice#TC_002_005_002" name="TC_002_005_002#AjxHotLoadOuterDevice#
端外拉取云化bundle" ajax_test_type="5" timeoutInMillis="1800000"
description="002_005_002[端外拉取云化bundle]" priority="P0"
platform="android,ios" groupName="高德"
scriptJar="AmapContainerArchTotoro-1.0.0-SNAPSHOT-tests.jar"/>

        <!-- ... -->
    </test-cases>
    <test-cases name="AjxHotLoadDelayLoadTest">
        <test-case summary="003_001" type="totoro" bizline="高
德" author="雪平"
atapCaseMethodName="com.totoro.auto.testcases.ajax.hotload.test.AjxHotLoa
dDelayLoadTest#TC_003_001" name="TC_003_001#AjxHotLoadDelayLoadTest#app
延迟生效-退页面生效动态引擎测试" ajax_test_type="5"
timeoutInMillis="1800000" description="003_001[app延迟生效-退页面生效动态
引擎测试]" priority="P0" platform="android,ios" groupName="高德"
scriptJar="AmapContainerArchTotoro-1.0.0-SNAPSHOT-tests.jar"/>

        <!-- ... -->
    </test-cases>

```

```

        <test-cases name="AjxHotLoadRollbackTest">
            <test-case summary="004_006" type="totoro" bizline="高德" author="雪平"
atapCaseMethodName="com.totoro.auto.testcases.ajax.hotload.test.AjxHotLoadRollbackTest#TC_004_006" name="TC_004_006#AjxHotLoadRollbackTest#ajax bundle回滚动态引擎测试" ajax_test_type="5" timeoutInMillis="1800000"
description="004_006[ajax bundle回滚动态引擎测试]" priority="P0"
platform="android,ios" groupName="高德"
scriptJar="AmapContainerArchTotoro-1.0.0-SNAPSHOT-tests.jar"/>
            <!-- ... -->
        </test-cases>
        <test-cases name="AjxHotStartService">
            <test-case summary="006_001" type="totoro" bizline="高德" author="宝森"
atapCaseMethodName="com.totoro.auto.testcases.ajax.hotload.test.AjxHotStartService#TC_006_001" name="TC_006_001#AjxHotStartService#启动js service" ajax_test_type="5" timeoutInMillis="1800000"
description="006_001[启动js service]" priority="P0"
platform="android,ios" groupName="高德"
scriptJar="AmapContainerArchTotoro-1.0.0-SNAPSHOT-tests.jar"/>
            <!-- ... -->
        </test-cases>
        <test-cases name="AjxStartServiceHotLoad">
            <test-case summary="007_001" type="totoro" bizline="高德" author="宝森"
atapCaseMethodName="com.totoro.auto.testcases.ajax.hotload.test.AjxStartServiceHotLoad#TC_007_001" name="TC_007_001#AjxStartServiceHotLoad#启动ajax service时获取更新" ajax_test_type="5" timeoutInMillis="1800000"
description="007_001[启动ajax service时获取更新]" priority="P0"
platform="android,ios" groupName="高德"
scriptJar="AmapContainerArchTotoro-1.0.0-SNAPSHOT-tests.jar"/>
            <!-- ... -->
        </test-cases>
        <test-cases name="AjxHotLoadBaseLineTemplateTest">
            <test-case summary="009_001_001" type="totoro"
bizline="高德" author="宝森"
atapCaseMethodName="com.totoro.auto.testcases.ajax.hotload.test.AjxHotLoadBaseLineTemplateTest#TC_009_001_001"
name="TC_009_001_001#AjxHotLoadBaseLineTemplateTest#随版同时生效-未配置skipRefresh-进入随包bundle" ajax_test_type="5" timeoutInMillis="1800000"
description="009_001_001[随版同时生效-未配置skipRefresh-进入随包bundle]"
priority="P0" platform="android,ios" groupName="高德"
scriptJar="AmapContainerArchTotoro-1.0.0-SNAPSHOT-tests.jar"/>
            <!-- ... -->
        </test-cases>

```

附加执行包表示当前用例集使用哪一个执行包来执行；
执行指令表示执行当前用例集时的执行包执行指令，通过该指令来执行执行包的testNG用例；

3. 测试项

126 AJK动态热更新自动化测试 AJK动态热更新自动化测试 朱宝森 2022-01-27 10:36:52 朱宝森

名称: AJK动态热更新自动化测试

测试类型: 功能测试

描述: AJK动态热更新自动化测试

执行位置: ☒ 客户端与设备 ☐ 仅客户端 ☐ 仅设备

操作系统: ☒ Windows ☒ Android ☒ Mac ☒ iOS ☒ Linux

测试环境(分钟): 1440

业务负责人: 朱宝森(更新)-266072

任务名称: 更新

结果展示模板: TestNG模板执行

结果看板展示: ☒ 是 ☐ 否

标签: + 标签

业务线标签: + 标签

ID	名称	描述	创建人	详情
3	Android应用安装	应用安装、安装插件	吴建英	详情
4	iOS应用安装	应用安装、安装	吴建英	详情
5	环境搭建	环境搭建、设备准备与配置	吴建英	详情
6	SLAM测试任务迁移模板展示	执行TestNG测试、上传结果	吴建英	详情
9	多渠道分发插件、管理端头	多渠道分发插件、管理端头	韩海	详情
12	兼容性测试用例集	兼容性测试用例集	王坤	详情
16	性能测试用例集-Android-UI	性能测试用例集	王坤	详情
24	性能测试用例集	性能测试用例集	王坤	详情
27	SLAM测试任务迁移模板展示	SLAM测试任务迁移模板展示、上传结果	吴建英	详情
28	SLAM测试任务迁移模板展示	SLAM测试任务迁移模板展示、上传结果	吴建英	详情

4. 测试任务

ID	名称	描述	创建人	创建时间	标签	操作
7784	apk bundle热更新自动化测试_android	apk bundle热更新自动化测试_android	朱宝森	2022-03-28 11:47:22		详情 编辑 执行 指定执行 历史 更多...
7783	apk bundle热更新自动化测试_ios	apk bundle热更新自动化测试_ios	朱宝森	2022-03-28 11:45:46		详情 编辑 执行 指定执行 历史 更多...

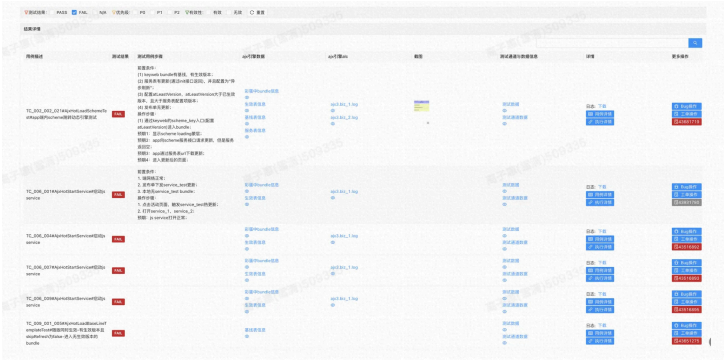
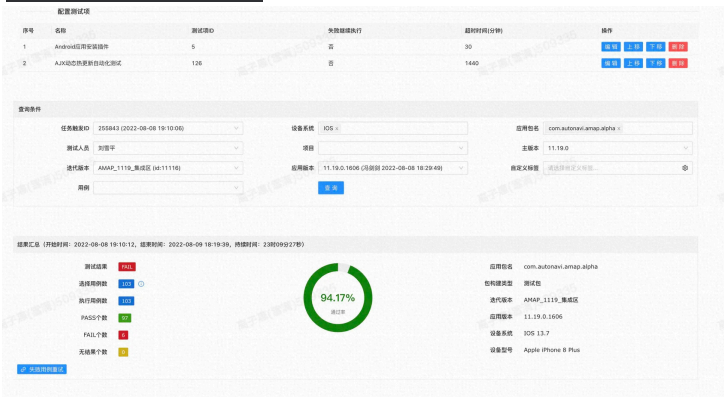
测试任务中包含两个测试项，分别是APP安装插件与ajx_bundle热更新自动化：

序号	名称	测试ID	失败继续执行	超时时间(分钟)	操作
1	Android应用安装插件	5	否	30	详情 编辑 删除 置顶 取消置顶
2	AJK动态热更新自动化测试	126	否	1440	详情 编辑 删除 置顶 取消置顶

3. 结果与展望

3.1 测试结果

测试结果页面如下：



工单的自动处理如下：

 复制代码

```
data.autoBugReport = false;
if (data.result === 'FAIL'){
  let hasMonkeyError = _.get(data, "extend.hasMonkeyError");
  if (true !== hasMonkeyError){
    data.autoBugReport = true;
  }
}
return data;
```

3.2 收益

1. 测试效率提升显著

按照梵天平台bundle构建与创建发布单，测试demo拉取bundle编译、扫码运行，查看彩蛋、查看服务表、生效表基线表的步骤，手动测试的测试用时计算规则如下：

- bundle构建与发布单创建：按照一条用例用时大于等于20分钟；
- 测试脚本的编译与扫码运行：按照一条用例用时5分钟；
- 查看彩蛋、查看服务表、生效表基线表的步骤：按照一条用例用时大于等于5分钟；

测试项	用例数	手动执行耗时	单设备自动化执行耗时 (iOS极端场景)
-----	-----	--------	-------------------------

1. 维护成本

- 测试脚本：端内测试脚本与Totoro测试脚本迭代了3个大版本，基于上比较稳定，基本上无维护成本。但是，当APP欢迎页与手机权限页面发生变化，只需要适配相关的UI变动即可；
- 测试工具：除AJX Bundle热更新自动化外，AJX接口自动化测试、AJX组件自动化测试、AJX页面生命周期自动化测试、vmap自动化测试、AJX的iGate巡检共用一个测试工具，目前也比较稳定，分层也比较清晰，当ATAP二方库有接口变动时需要进行适配，除此之外，基本上不需要变动；

2. 任务触发

- 手动触发：只需要找到在ATAP平台上配置的任务，点击“执行”或者“自定义执行”，即可一键触发；
- 自定义定时触发：可以在ATAP平台上提前配置AJX Bundle的热更新自动化测试任务，并执行触发时间；

3.4 展望

- 测试项前置，基于titan提测迭代区进行自动化执行，提前暴露问题；
- 配置用例多设备分布式执行，极大的缩短执行时间；